Bitácora S08

Que la transferencia no deje dinero en el aire: saga orquestada y circuit breaker

Sistemas Distribuidos 2027-1 · Grupo 2

Dato	Detalle
Alumna	Alejandra Melissa Saucedo Serralde
Práctica	Lab S08: saga orquestada en `transferencia` (Parte 1) y circuit breaker en las llamadas a `cuenta` (Parte 2), con la Parte 0 para reproducir el problema
Fechas	Parte 0: 18 de octubre de 2026 (por la noche). Saga y circuit breaker: 23 de octubre de 2026.
Horas	Los logs de Docker y las fechas guardadas en la base están en UTC; mi hora local es UTC−6 (04:43 UTC = 22:43 local).

Resultados de un vistazo

Qué probé	Resultado
Parte 0: el problema	Saldos 1000 + 500 = 1500 antes; 800 + 500 = 1300 después. **200 en el aire.**
Compensación con `cuenta` arriba	Saldos iguales antes y después (800 y 500). Respuesta `502`, estado `COMPENSADA`.
Compensación que también falla	Estado `COMPENSACION_PENDIENTE`; origen en 600 (200 retenidos). Con el reintento volvió a 800.
Compensación repetida	Segunda llamada: misma respuesta y saldo en 800 (no devuelve dos veces).
Circuit breaker	`CLOSED` → `OPEN` (con 4 fallos) → `HALF_OPEN` (10 s después) → `CLOSED` (con 2 pruebas buenas).
Pruebas unitarias	7 de 7 en verde (`TransferenciaServiceTest`).

1. El problema

Antes de tocar código reproduje lo que dice el README. Creé dos cuentas por el gateway: **Origen S08** con 1000 y **Destino S08** con 500.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ curl -s http://localhost:8080/api/cuentas/002180009160741201; echo; curl -s http://localhost:8080/
api/cuentas/002180009160741202; echo
{"id":4,"clabe":"002180009160741201","titular":"Origen S08","saldo":1000.00}
{"id":5,"clabe":"002180009160741202","titular":"Destino S08","saldo":500.00}
```

Figura 1. *Saldos antes: Origen 1000 y Destino 500.*

Levanté `transferencia` con `PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000`, lancé una transferencia de 200 y, a los 3 segundos (dentro de la pausa, ya hecho el cargo), apagué `cuenta`. El `docker compose stop` paró las tres réplicas.

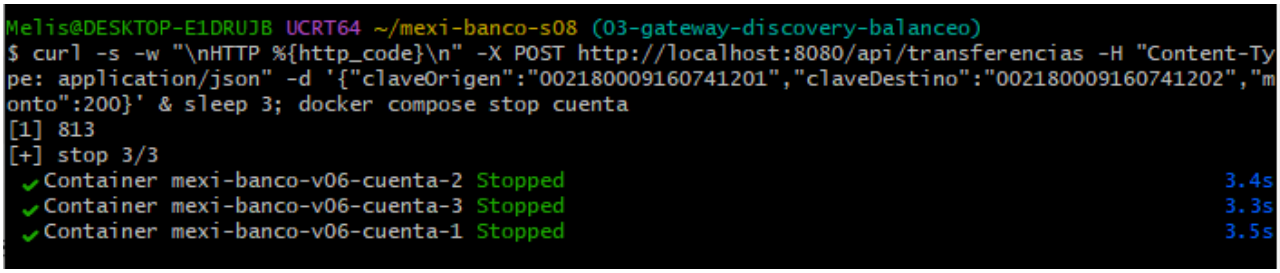


Figura 2. *La transferencia de 200 corre en segundo plano (`[1] 813`) y `cuenta` se detiene en sus tres réplicas.*

La respuesta del cliente fue un `504 Gateway Timeout` del gateway, sin ninguna explicación del dinero. Por dentro, `transferencia` terminó la pausa y falló al abonar con `IllegalStateException: No instances available for cuenta` (a las 20:56:39 hora local): Eureka ya no tenía instancias de `cuenta`.

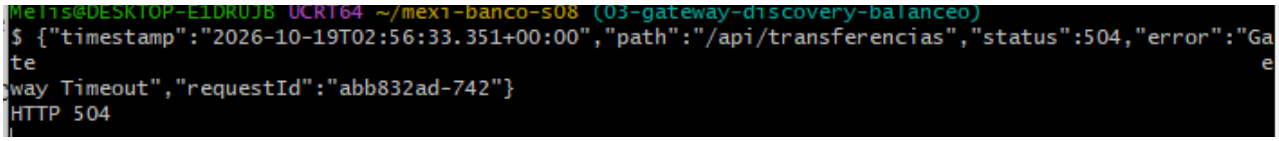


Figura 3. *Respuesta al cliente: `504 Gateway Timeout` (el mismo terminal de la figura anterior, unos segundos después).*

Volví a encender `cuenta` y consulté los saldos.

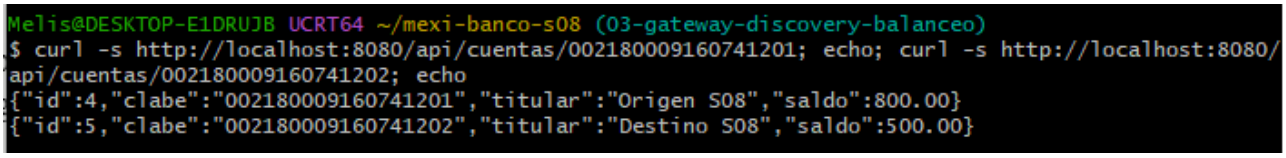


Figura 4. *Saldos después: Origen 800 y Destino 500. Antes sumaban 1500; ahora suman 1300.*

> **Por qué pasó:** el cargo y el abono son dos peticiones HTTP independientes y cada una se confirma sola en `cuenta`. Cuando el abono falló, no había ninguna transacción que cubriera a las dos para poder deshacer el cargo: el `@Transactional` del monolito ya no alcanza cuando la operación cruza servicios. El cargo quedó confirmado y nadie lo devolvió.

2. Mi saga

`TransferenciaService` ahora es el **orquestador** de una saga: decide qué paso sigue, guarda el estado de la transferencia en su tabla después de cada paso y, si algo falla después del cargo, ejecuta la compensación. Los estados son seis: `INICIADA`, `CARGADA`, `COMPLETADA`, `COMPENSADA`, `COMPENSACION_PENDIENTE` y `FALLIDA`.

#	Qué hace	Estado que guarda	Si falla
1	Valida: las cuentas son distintas y existen (consulta origen y destino)	(ninguno)	Responde el error de `cuenta`. No se movió dinero.
2	Registra la transferencia antes de tocar dinero	`INICIADA`	—

#	Qué hace	Estado que guarda	Si falla
3	Carga el monto a la cuenta origen	`CARGADA`	Si lo rechazan (saldo insuficiente, por ejemplo): `FALLIDA` y el error original llega al cliente. No hay nada que compensar.
4	Abona el monto a la cuenta destino	`COMPLETADA`	Se compensa (paso 5).
5	**Compensación:** deposita el monto de vuelta al origen y registra un movimiento `DEPOSITO` con referencia "compensacion transferencia N"	`COMPENSADA`	Si el depósito también falla: `COMPENSACION_PENDIENTE`.
6	Registra los dos movimientos y manda las notificaciones	(ya es `COMPLETADA`)	Solo se deja en el log: el dinero ya se movió y responder error sería engañoso.

Esta es la máquina de estados de una transferencia. En naranja, la compensación; en rojo, cuando la compensación también falla.

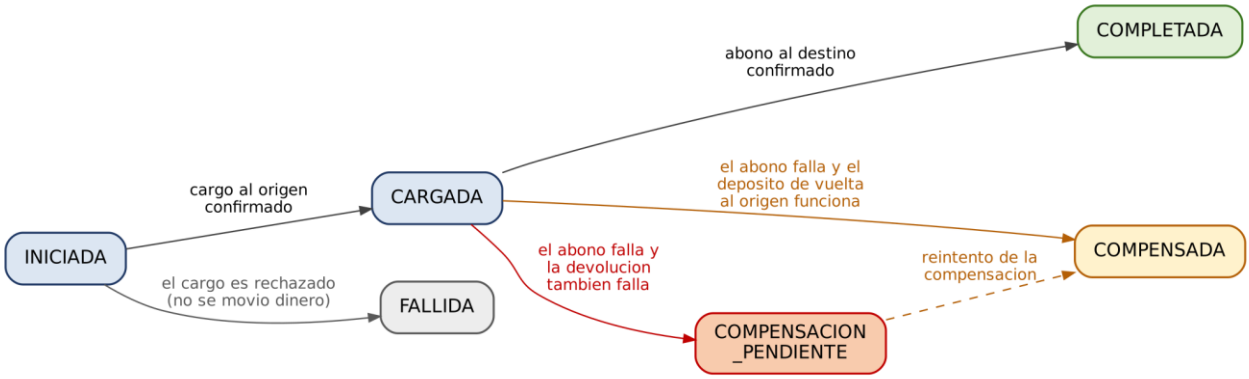


Figura 5. *Diagrama de estados de la transferencia.*

Cómo provoco la falla del abono sin apagar nada

Agregué la variable de entorno **FALLAR_ABONO** (propiedad `mexibanco.demo.fallar-abono`, declarada en el `application.yml` de `transferencia` y pasada por el `docker-compose.yml`; vale `false` por defecto). Con `FALLAR_ABONO=true`, el orquestador hace fallar el abono sin llamar a `cuenta`, de modo que `cuenta` sigue arriba y la compensación sí puede ejecutarse:

```
FALLAR_ABONO=true docker compose up -d transferencia
```

Para el caso en que la compensación también falla se usa la misma pausa de la Parte 0 (`PAUSA_ENTRE_CARGO_Y_ABONO_MS`) y se apaga `cuenta` durante ella.

Qué responde ahora el servicio

Ya no es un `500` sin explicación. Cuando la saga no se completa, `transferencia` responde **502 Bad Gateway** (la falla viene de un servicio del que depende) en formato Problem Details, con el id de la transferencia y el estado en que quedó, para que el cliente sepa qué pasó con su dinero y cómo consultarlo:

```
{"title":"Transferencia no completada","status":502,
 "detail":"No se pudo abonar al destino; el cargo fue devuelto a la cuenta origen
            (transferencia 2)","transferenciaId":2,"estado":"COMPENSADA"}
```

3. El código clave

3.1 La entidad guarda el estado

A `Transferencia` le agregué `estado` y `motivo` (por qué falló, si falló). El `default 'COMPLETADA'` en la columna es para las filas que ya existían desde el S07: la base tenía una transferencia anterior, y sin el default Postgres no deja agregar una columna `NOT NULL` a una tabla con datos.

```
@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 30,
        columnDefinition = "varchar(30) default 'COMPLETADA'")
private EstadoTransferencia estado = EstadoTransferencia.INICIADA;

@Column(length = 500)
private String motivo;
```

3.2 El orquestador: un estado guardado por paso

`transferir()` guarda el estado después de cada paso. No tiene `@Transactional` a propósito: cada `save()` es su propia transacción local y queda confirmado en la base al instante. Así, si `transferencia` se cayera a la mitad, la tabla diría en qué paso se quedó. Una transacción de base de datos no podría deshacer el cargo de todos modos, porque el dinero vive en otro servicio.

```
// Paso 1: dejar constancia antes de tocar dinero
Transferencia transferencia =
        transferencias.save(new Transferencia(claveOrigen, claveDestino, monto));

// Paso 2: cargo (si lo rechazan: FALLIDA y se relanza el error original)
var cargo = cargarOMarcarFallida(transferencia, claveOrigen, monto);
transferencia.cambiarEstado(EstadoTransferencia.CARGADA, null);
transferencias.save(transferencia);

// Paso 3: abono; si falla, abonarODeshacer() compensa y lanza
// TransferenciaFallidaException
var abono = abonarODeshacer(transferencia, claveOrigen, claveDestino, monto);
transferencia.cambiarEstado(EstadoTransferencia.COMPLETADA, null);
transferencias.save(transferencia);
```

3.3 La compensación

La compensación **no es un rollback**: es una operación nueva, un depósito a la cuenta origen por el mismo monto. Fíjate en el orden: en cuanto el depósito se confirma, se guarda `COMPENSADA` (antes de registrar el movimiento o de hacer cualquier otra cosa), para que una compensación ya hecha no se vuelva a aplicar. Si el depósito falla, queda `COMPENSACION_PENDIENTE` con los dos errores en `motivo`.

```
try {
        devuelta = cuentas.abonar(claveOrigen, monto);   // deposito de vuelta al origen
} catch (RuntimeException falloCompensacion) {
        transferencia.cambiarEstado(EstadoTransferencia.COMPENSACION_PENDIENTE,
                motivo + " | Compensacion fallida: " + falloCompensacion.getMessage());
        transferencias.save(transferencia);
        return new TransferenciaFallidaException(transferencia.getId(),
                "COMPENSACION_PENDIENTE", "...requiere intervencion...", falloCompensacion);
}
transferencia.cambiarEstado(EstadoTransferencia.COMPENSADA, motivo);  // se guarda ya
transferencias.save(transferencia);
movimientos.registrar(claveOrigen, TipoMovimiento.DEPOSITO, monto, devuelta.saldo(),
        "compensacion transferencia " + transferencia.getId());
```

3.4 La respuesta al cliente

`TransferenciaFallidaException` lleva el id y el estado. El `ManejadorDeErrores` (el único lugar donde una excepción se vuelve respuesta HTTP) la convierte en el `502` de la sección 2:

```
@ExceptionHandler(TransferenciaFallidaException.class)
ProblemDetail transferenciaFallida(TransferenciaFallidaException e) {
        ProblemDetail problema = ProblemDetail.forStatusAndDetail(
                HttpStatus.BAD_GATEWAY, e.getMessage());
        problema.setTitle("Transferencia no completada");
        problema.setProperty("transferenciaId", e.getTransferenciaId());
        problema.setProperty("estado", e.getEstado());
        return problema;
}
```

3.5 Reintentar una compensación pendiente (idempotente)

Para terminar las transferencias que quedan en `COMPENSACION_PENDIENTE` agregué `POST /transferencias/{id}/compensacion` (y su ruta en el gateway, porque la ruta original solo dejaba pasar `/api/transferencias` y `/api/transferencias/{id}`). La clave está en las dos primeras reglas: si ya está `COMPENSADA` no hace nada, y si no está pendiente responde `409`.

```
if (transferencia.getEstado() == EstadoTransferencia.COMPENSADA) {
        return transferencia;          // ya compensada: no se hace nada
}
if (transferencia.getEstado() != EstadoTransferencia.COMPENSACION_PENDIENTE) {
        throw new ConflictoException("...solo se compensan las que estan en
                COMPENSACION_PENDIENTE");
}
var devuelta = cuentas.abonar(transferencia.getClaveOrigen(), transferencia.getMonto());
transferencia.cambiarEstado(EstadoTransferencia.COMPENSADA, "Compensacion reintentada...");
transferencias.save(transferencia);
```

4. La compensación funcionando

Levanté `transferencia` con `FALLAR_ABONO=true` y `cuenta` arriba, y corrí tres pasos en una sola terminal: saldos antes, una transferencia de 200 y saldos después.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ echo "--- ANTES ---"; curl -s http://localhost:8080/api/cuentas/002180009160741201; echo; curl -s
http://localhost:8080/api/cuentas/002180009160741202; echo
--- ANTES ---
{"id":4,"clabe":"002180009160741201","titular":"Origen S08","saldo":800.00}
{"id":5,"clabe":"002180009160741202","titular":"Destino S08","saldo":500.00}

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ echo "--- TRANSFERENCIA ---"; curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:8080/api
/transferencias -H "Content-Type: application/json" -d '{"claveOrigen":"002180009160741201","claveDe
stino":"002180009160741202","monto":200}'
--- TRANSFERENCIA ---
{"type":"about:blank","title":"Transferencia no completada","status":502,"detail":"No se pudo abonar
 al destino; el cargo fue devuelto a la cuenta origen (transferencia 2)","instance":"/transferencias
","transferenciaId":2,"estado":"COMPENSADA"}
HTTP 502

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ echo "--- DESPUES ---"; curl -s http://localhost:8080/api/cuentas/002180009160741201; echo; curl -
s http://localhost:8080/api/cuentas/002180009160741202; echo
--- DESPUES ---
{"id":4,"clabe":"002180009160741201","titular":"Origen S08","saldo":800.00}
{"id":5,"clabe":"002180009160741202","titular":"Destino S08","saldo":500.00}
```

Figura 6. *Antes: 800 y 500. La transferencia responde `502` con `estado: COMPENSADA` (transferencia 2). Después: 800 y 500.*

Los saldos de origen y destino son **idénticos antes y después**: el cargo de 200 se hizo y la compensación lo devolvió. La respuesta ya explica qué pasó y trae el id de la transferencia. Con ese id consulté el estado guardado en la base:

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ curl -s http://localhost:8080/api/transferencias/2; echo
{"id":2,"claveOrigen":"002180009160741201","claveDestino":"002180009160741202","monto":200.00,"fecha
":"2026-10-24T03:28:20.191959Z","estado":"COMPENSADA","motivo":"Abono fallido: Fallo inyectado en el
 abono (mexibanco.demo.fallar-abono=true)"}
```

Figura 7. *`GET /api/transferencias/2: estado: COMPENSADA` y el `motivo` guarda la causa (el fallo inyectado en el abono).*

Además dejé pruebas unitarias de este camino (Anexo A): un abono que falla se compensa con un solo depósito al origen, y se guarda el movimiento `DEPOSITO` con la referencia de compensación.

5. Cuando la compensación también falla

Repetí la Parte 0 con el código nuevo: `FALLAR_ABONO=false`, pausa de 10 segundos y `cuenta` apagada a los 2 segundos. Ahora ni el abono ni la devolución pueden llegar a `cuenta`.

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ {"type":"about:blank","title":"Transferencia no completada","status":502,"detail":"No se pudo abon
ar                                                                                              r
 al destino ni devolver el cargo al origen: el dinero esta retenido y requiere intervencion (transfe
rencia 3)","instance":"/transferencias","transferenciaId":3,"estado":"COMPENSACION_PENDIENTE"}
HTTP 502

[1]+  Done                    curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:8080/ap
i/transferencias -H "Content-Type: application/json" -d '{"claveOrigen":"002180009160741201","claveD
estino":"002180009160741202","monto":200}'
```

Figura 8. *La respuesta explica que el dinero quedó retenido: `estado: COMPENSACION_PENDIENTE` (transferencia 3).*

Con `cuenta` otra vez arriba revisé el dinero y la transferencia. El origen quedó en **600**: los 200 siguen retenidos (el cargo se hizo y no se devolvió). El `motivo` guarda los dos errores, el del abono y el de la compensación.

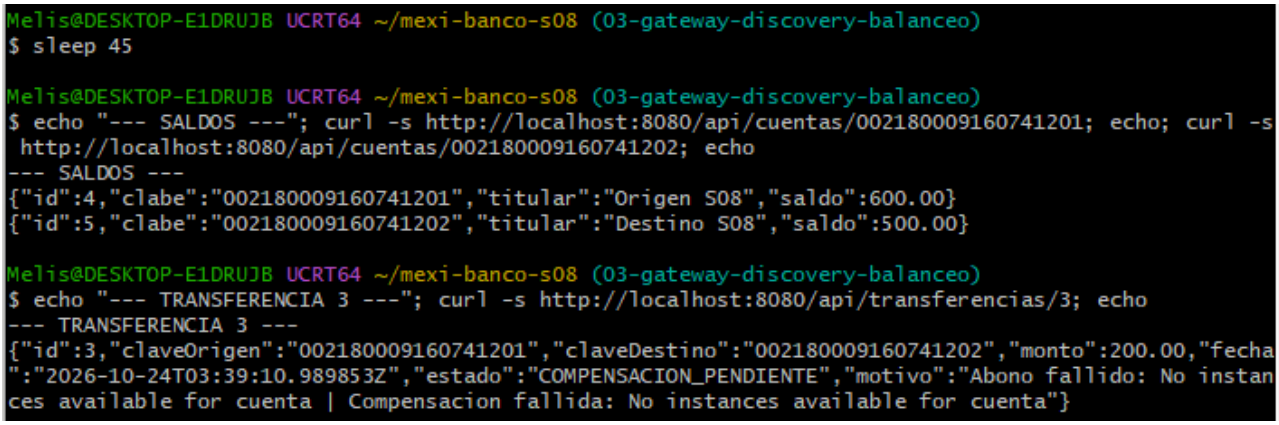


Figura 9. *Origen 600, Destino 500, y la transferencia 3 en `COMPENSACION_PENDIENTE` con ambos errores en `motivo`.*

La transferencia ya no se queda en `CARGADA` para siempre: el estado dice exactamente qué falta hacer.

Cómo la termino

- **Lo que implementé:** el endpoint `POST /transferencias/{id}/compensacion`, que reintenta el depósito a mano. En la captura, la primera llamada devolvió la transferencia a `COMPENSADA` y el origen subió de 600 a 800.
- **Lo que haría en producción** (no lo implementé): un proceso programado (`@Scheduled`) que cada cierto tiempo busque las transferencias en `COMPENSACION_PENDIENTE` y llame a esa misma operación, con una espera creciente entre intentos (1 s, 2 s, 4 s…) y un tope de intentos, después del cual avise a una persona.



Figura 10. *Compensación reintentada dos veces: la primera sube el origen de 600 a 800; la segunda devuelve lo mismo y el saldo sigue en 800.*

Por qué la compensación debe ser idempotente

Un reintento ocurre justo cuando no sabemos si el intento anterior sí se aplicó. Si la compensación no fuera idempotente, cada reintento volvería a depositar el monto y le devolveríamos al cliente más dinero del que le habíamos cobrado: **reintentar sin idempotencia cobra (o paga) dos veces**. En mi versión la idempotencia sale del estado: solo se deposita desde `COMPENSACION_PENDIENTE`, y `COMPENSADA` se guarda en cuanto el depósito se confirma. Por eso la segunda llamada de la captura no tocó `cuenta` y regresó la misma respuesta.

> **Limitación que reconozco:** esta idempotencia es parcial. Si el depósito se aplica en `cuenta` pero la respuesta se pierde antes de que yo guarde `COMPENSADA`, o si dos reintentos corren al mismo tiempo, se podría depositar dos veces. Lo robusto es mandarle a `cuenta` una llave de idempotencia junto con la compensación (como hace el SPEI del S07 con `Idempotency-Key`), para que sea `cuenta` quien ignore el segundo depósito. No lo hice y queda como mejora.

6. El circuit breaker

Puse un circuit breaker de **Resilience4j** en las llamadas de `transferencia` a `cuenta`. Si `cuenta` está caída, después de unos fallos el circuito se **abre** y las llamadas se rechazan al instante en lugar de seguir golpeando a un servicio muerto o quedarse esperando el timeout. Pasado un tiempo se pone **semiabierto**, deja pasar unas pruebas y, si salen bien, se **cierra**.

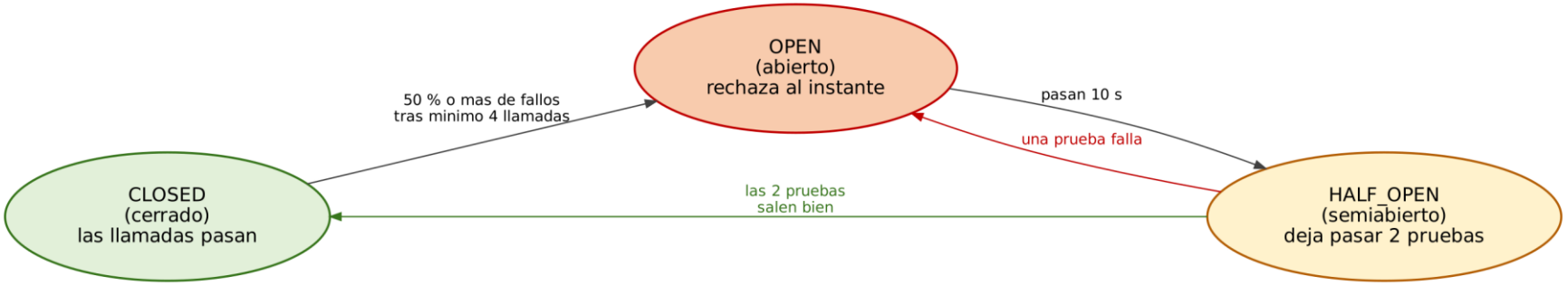


Figura 11. *Los tres estados del breaker y sus transiciones, con mis parámetros.*

Parámetro	Valor	Qué significa
`slidingWindowSize`	6	Mira las últimas 6 llamadas.
`minimumNumberOfCalls`	4	No decide nada antes de 4 llamadas.
`failureRateThreshold`	50 %	Si la mitad o más de la ventana falló, abre.
`waitDurationInOpenState`	10 s	Tiempo que se queda abierto antes de probar.
`automaticTransitionFromOpenToHalfOpenEnabled`	true	Pasa solo a semiabierto al cumplirse los 10 s, sin esperar una llamada.
`permittedNumberOfCallsInHalfOpenState`	2	En semiabierto deja pasar 2 pruebas.
`recordFailurePredicate`	`FalloDeInfraestructura`	Solo cuentan como fallo los errores de infraestructura: sin conexión, sin instancias en Eureka o respuestas 5xx. Un `404` o un `422` (saldo insuficiente) son respuestas normales de negocio y **no** abren el circuito.

Cómo está puesto en el código

`CuentaCliente` lleva `@CircuitBreaker(name = "cuenta")` a nivel de clase, así que sus tres llamadas (`consultar`, `cargar` y `abonar`) pasan por el mismo breaker. Los parámetros viven en el `application.yml`, y un `ManejadorDeErrores` extendido responde `503` con un mensaje claro cuando el circuito está abierto o no hay instancias de `cuenta`. Una clase pequeña deja en el log cada cambio de estado (`CircuitBreakerLogs`).

```
@CircuitBreaker(name = "cuenta")
@Component
public class CuentaCliente { ... }

resilience4j:
  circuitbreaker:
    instances:
      cuenta:
        slidingWindowSize: 6
        minimumNumberOfCalls: 4
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
        automaticTransitionFromOpenToHalfOpenEnabled: true
        permittedNumberOfCallsInHalfOpenState: 2
        recordFailurePredicate: mx.mexibanco.clientes.FalloDeInfraestructura
```

Nota sobre la dependencia: el README menciona `spring-cloud-starter-circuitbreaker-resilience4j`. Yo usé `io.github.resilience4j:resilience4j-spring-boot3` (versión 2.2.0, fijada a mano porque el BOM de Spring Cloud no la maneja) más `spring-boot-starter-aop`. Con ella se configura todo desde el `application.yml`, funciona la anotación `@CircuitBreaker` y se tiene el endpoint `/actuator/circuitbreakers`, que expuse en `management.endpoints.web.exposure.include`. No lo hice a mano.

Evidencia: los tres estados

Cerrado (estado inicial). `/actuator/circuitbreakers` antes de empezar: `state: CLOSED` y cero llamadas registradas.

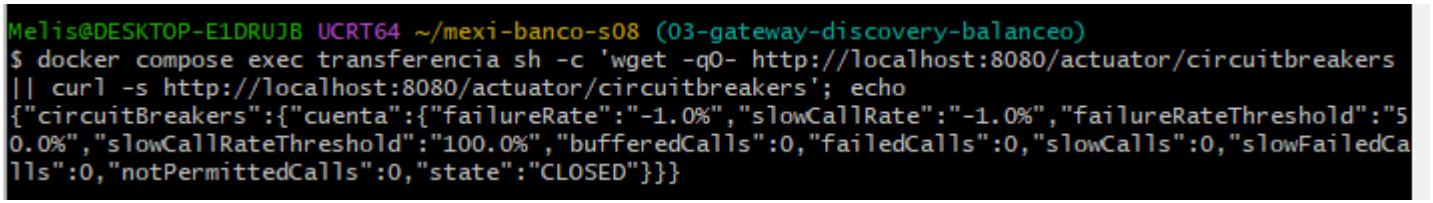


Figura 12. *Estado inicial del breaker `cuenta`: `CLOSED`.*

Cerrado → abierto. Apagué `cuenta` y mandé 6 transferencias seguidas. Los intentos 1 a 4 respondieron `503` "No hay instancias disponibles del servicio solicitado" (Eureka ya no tenía instancias de `cuenta`); cada uno contó como fallo, y con 4 llamadas y 100 % de fallos el breaker **abrió**. Los intentos 5 y 6 ya recibieron `503` con "El servicio cuenta no está disponible (circuito abierto)" y `circuito: ABIERTO`: se rechazaron sin intentar la llamada a `cuenta`.

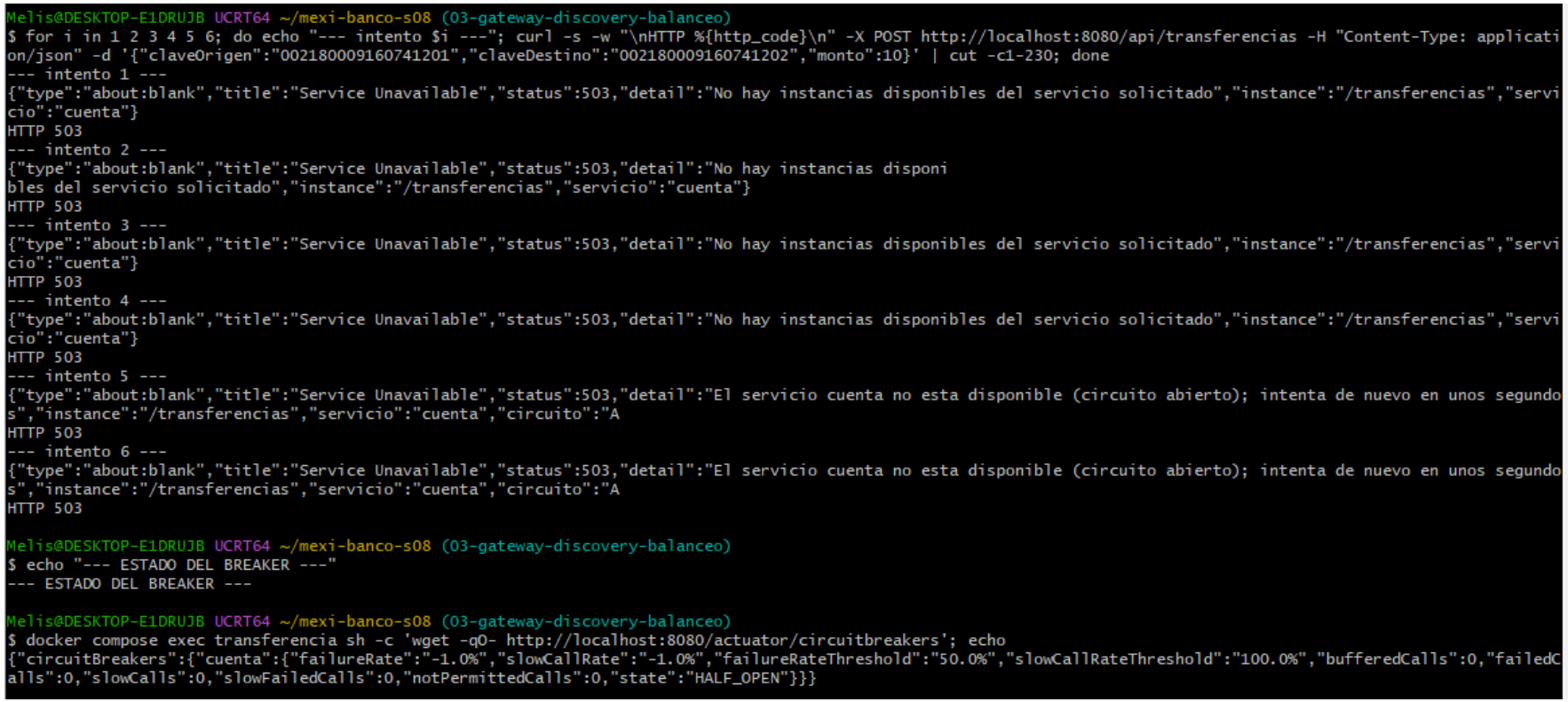


Figura 13. *Los 6 intentos con `cuenta` apagada: los primeros 4 fallan por falta de instancias, el 5 y el 6 se rechazan por circuito abierto. Al consultar el estado, ya habían pasado más de 10 s y marcaba `HALF_OPEN`.*

Abierto → semiabierto → cerrado. Encendí `cuenta` y esperé a que Eureka la volviera a registrar. El breaker seguía en `HALF_OPEN`. Mandé 2 transferencias de 10: las dos dieron `201` con estado `COMPLETADA`, y el breaker volvió a `CLOSED` con 0 % de fallos. Las dos primeras llamadas fueron las pruebas del semiabierto; las otras 6 ya contaron en el circuito cerrado (`bufferedCalls: 6`, `failedCalls: 0`).

```
Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ echo "--- CUENTA ---"; docker compose ps cuenta --format "table {{.Service}}\t{{.Status}}"
--- CUENTA ---
SERVICE   STATUS
cuenta    Up About a minute (healthy)
cuenta    Up About a minute (healthy)
cuenta    Up About a minute (healthy)

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ echo "--- BREAKER ANTES ---"; docker compose exec transferencia sh -c 'wget -qO- http://localhost:8080/act
uator/circuitbreakers'; echo
--- BREAKER ANTES ---
{"circuitBreakers":{"cuenta":{"failureRate":"-1.0%","slowCallRate":"-1.0%","failureRateThreshold":"50.0%","s
lowCallRateThreshold":"100.0%","bufferedCalls":0,"failedCalls":0,"slowCalls":0,"slowFailedCalls":0,"notPermi
ttedCalls":0,"state":"HALF_OPEN"}}}

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ for i in 1 2; do echo "--- transferencia $i ---"; curl -s -w "\nHTTP %{http_code}\n" -X POST http://localh
ost:8080/api/transferencias -H "Content-Type: application/json" -d '{"claveOrigen":"002180009160741201","cla
veDestino":"002180009160741202","monto":10}' | cut -c1-230; done
--- transferencia 1 ---
{"id":4,"claveOrigen":"002180009160741201","claveDestino":"002180009160741202","monto":10,"fecha":"2026-10-2
4T04:47:46.177516183Z","estado":"COMPLETADA","motivo":null}
HTTP 201
--- transferencia 2 ---
{"id":5,"claveOrigen":"002180009160741201","claveDestino":"002180009160741202","monto":10,"fecha":"2026-10-2
4T04:47:48.435253590Z","estado":"COMPLETADA","motivo":null}
HTTP 201

Melis@DESKTOP-E1DRUJB UCRT64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ echo "--- BREAKER DESPUES ---"; docker compose exec transferencia sh -c 'wget -qO- http://localhost:8080/a
ctuator/circuitbreakers'; echo
--- BREAKER DESPUES ---
{"circuitBreakers":{"cuenta":{"failureRate":"0.0%","slowCallRate":"0.0%","failureRateThreshold":"50.0%","slo
wCallRateThreshold":"100.0%","bufferedCalls":6,"failedCalls":0,"slowCalls":0,"slowFailedCalls":0,"notPermitt
edCalls":0,"state":"CLOSED"}}}
```

Figura 14. *`HALF_OPEN` antes, dos transferencias exitosas (`201`, `COMPLETADA`) y `CLOSED` después.*

Los logs de `transferencia` dejan la secuencia con su hora exacta y el conteo de llamadas rechazadas con el circuito abierto (2):

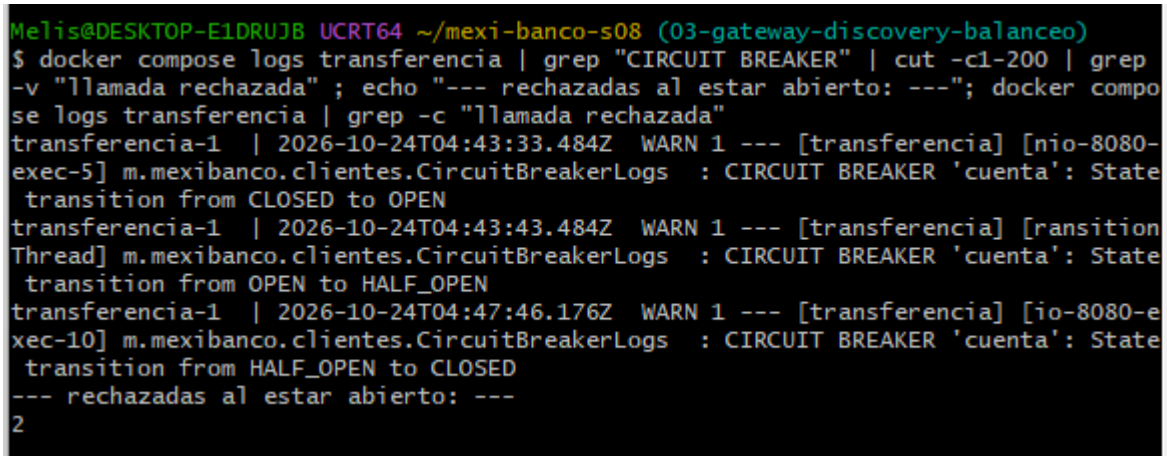


Figura 15. *Transiciones registradas en el log y 2 llamadas rechazadas con el circuito abierto.*

Hora (UTC)	Hora local	Transición
04:43:33.484	22:43:33	`CLOSED` → `OPEN` (4 fallos seguidos)
04:43:43.484	22:43:43	`OPEN` → `HALF_OPEN`: exactamente 10 s después, como configuré
04:47:46.176	22:47:46	`HALF_OPEN` → `CLOSED`: con la primera transferencia después de encender `cuenta`

Lo que no hice: los dos opcionales. No agregué un timeout nuevo (siguen los del S07: 2 s para conectar y 5 s para leer) ni el reintento con espera creciente de la compensación. El breaker comparte el mismo circuito para las llamadas normales y para la compensación: si está abierto, una compensación también se rechaza al instante y la transferencia queda en `COMPENSACION_PENDIENTE`, que es justo para lo que está ese estado.

7. Lo que la saga no te da

La saga me da **atomicidad aparente**: al final, o la transferencia se completa o se compensa. Lo que no me da es **aislamiento** (la "I" de ACID). Entre el cargo y la compensación, el estado intermedio es visible para cualquiera.

Esto lo vi en mis propias pruebas. Al final de la Parte 0, y mientras la transferencia 3 estuvo en `COMPENSACION_PENDIENTE`, quien consultaba el saldo del origen veía **800 y luego 600, cuando el saldo real era 1000 y 800**, y el destino no había recibido nada: los dos saldos sumaban 200 menos que el dinero que realmente hay en el banco. El dinero "estaba en el aire" y nadie, salvo mi tabla de estados, lo sabía. Lo mismo ocurre, en menor tiempo, entre el cargo y la compensación de una saga que sí se compensa: la ventana es corta, pero existe.

Eso tiene consecuencias reales. Alguien que lea ese saldo bajo puede **tomar una decisión equivocada**: rechazar un pago porque "no alcanza", calcular un límite de crédito o mostrarle al cliente un saldo que luego cambiará. Y aunque la compensación termine bien, esa decisión ya se tomó con un dato que no era verdadero. En el monolito esto no pasaba porque la transacción de base de datos escondía los cambios hasta el commit.

Lo que se hace en una saga para reducirlo es aceptar el estado intermedio y diseñarlo: por ejemplo, un **bloqueo semántico** (marcar el dinero como "reservado" o "transferencia en curso" para que quien consulte sepa que ese saldo es provisional), o leer el saldo a través del estado de la transferencia. Yo no implementé ninguna de las dos; mi solución solo garantiza que el sistema termine consistente, no que nadie lo vea en medio.

Anexo A. Pruebas unitarias

Reescribí `TransferenciaServiceTest`: el constructor ahora recibe el interruptor de falla, y la prueba que documentaba el bug del S07 ("si el abono falla, nadie deshace el cargo") la sustituí por las de la saga. Son 7 pruebas: 3 del S07 y 4 nuevas (compensación exitosa, compensación fallida, cargo rechazado y fallo inyectado). Las corrí con Maven dentro de un contenedor:

```
MSYS_NO_PATHCONV=1 docker run --rm -v "$(pwd -W)/transferencia:/app" -w /app \
    eclipse-temurin:21-jdk ./mvnw test -Dtest=TransferenciaServiceTest

[INFO] Running mx.mexibanco.transferencia.TransferenciaServiceTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.953 s
[INFO] Results:
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Anexo B. Qué cambié

Archivo	Cambio
`transferencia/.../EstadoTransferencia.java`	Nuevo: los seis estados de la saga.
`transferencia/.../Transferencia.java`	Campos `estado` y `motivo`, y `cambiarEstado()`.
`transferencia/.../TransferenciaService.java`	Orquestador de la saga, compensación y reintento de compensación.
`transferencia/.../TransferenciaController.java`	Nuevo `POST /{id}/compensacion`.
`transferencia/.../compartido/TransferenciaFallidaException.java`	Nuevo: lleva el id y el estado.
`transferencia/.../compartido/ManejadorDeErrores.java`	Respuestas `502` (saga fallida) y `503` (circuito abierto, sin instancias).
`transferencia/.../clientes/CuentaCliente.java`	`@CircuitBreaker(name = "cuenta")`.
`transferencia/.../clientes/FalloDeInfraestructura.java`	Nuevo: qué errores cuentan como fallo del breaker.
`transferencia/.../clientes/CircuitBreakerLogs.java`	Nuevo: log de cada transición de estado.
`transferencia/pom.xml`	Dependencias `resilience4j-spring-boot3` y `spring-boot-starter-aop`.
`transferencia/src/main/resources/application.yml`	`fallar-abono`, parámetros del breaker y endpoints de actuator.
`transferencia/src/test/.../TransferenciaServiceTest.java`	Pruebas de la saga.
`docker-compose.yml`	Variable `FALLAR_ABONO` para `transferencia`.

Archivo	Cambio
`gateway/src/main/resources/application.yml`	La ruta de `transferencia` ahora deja pasar `/api/transferencias/{id}/compensacion`.